

MagicQ Show-Computer auf Dell Wyse 3040

Ubuntu + Openbox + ChamSys MagicQ als dedizierter Show-PC für den **Compact Wing**, mit automatischem Mounten von USB-Sticks.

Wichtiger Hinweis zur Hardware: Der Wyse 3040 hat in vielen Ausführungen nur **2 GB RAM** und **8 GB eMMC**. MagicQ ist relativ ressourcenhungrig und der Compact Wing wird für „Unlocked“-Betrieb benötigt. Mit 2 GB RAM läuft alles knapp, aber funktioniert. Es wird empfohlen, **nur ein leichtgewichtiges System** (Server-Install + Openbox) zu betreiben und nichts weiter zu installieren.

1. Voraussetzungen / Hardware

Teil	Empfehlung
Wyse 3040	Atom x5-Z8350, 2 GB RAM, 8 GB eMMC, UEFI-only (kein Legacy-Boot)
USB-Stick	≥ 4 GB, UEFI-bootfähig (empfohlen: mit Ventoy vorbereiten)
Display	Dual DisplayPort → verwende aktiven DP→HDMI-Adapter (DP++ funktioniert oft nicht zuverlässig)
Compact Wing	per USB anschließen (kein Treiber nötig)

Eigenschaften des Wyse 3040:

- Speicher erscheint als `/dev/mmcblk0` (eMMC), nicht `/dev/sda`.
- Nur 64-Bit-UEFI-Boot möglich.
- Das eMMC-Gerät hat oft einen Namen mit einem Sonderzeichen (z. B. `MMC H8G4a\x92`), was Installer verwirren kann → siehe Abschnitt 3.

2. BIOS vorbereiten

1. Gerät einschalten und wiederholt **F2** drücken.
2. Falls gesperrt: BIOS mit Passwort `Fireport` (oder `Fireport2`) entsperren.
3. `General → Boot Sequence`: USB-Stick an erste Stelle setzen.
 - Merke dir den **File Name** der Boot-Option (meist `\EFI\BOOT\BOOTX64.EFI`).
 - Der Wyse benötigt später genau diesen Pfad auf der internen eMMC (siehe Abschnitt 4).
4. Optional: `Maintenance → Data Wipe` → `Wipe on Next Boot` aktivieren, um das eMMC zu löschen
(falls alte ThinOS-Installation vorhanden).

5. Speichern und neu starten.

3. Ubuntu installieren (empfohlen: 24.04 LTS Server)

*Wichtig: Der neuere Ubuntu-Installer (26.04) findet die eMMC teils nicht / friert ein. **Ubuntu 24.04 LTS funktioniert nachweislich.***

1. Ubuntu **22.04 oder 24.04 LTS (am besten Server-Image)** auf den USB-Stick bringen (z. B. mit Ventoy oder `rufus` / `dd`).
2. USB-Stick einstecken, Boot-Menü mit **F12** öffnen, vom USB-Stick booten.
3. **Server-Installation** wählen (keine Desktop-Oberfläche nötig – wir bauen Openbox selbst auf, das spart massiv RAM/Platz auf der kleinen eMMC).
4. Bei der Festplattenauswahl: Das eMMC (`/dev/mmcblk0`) gewählt. Falls es **nicht angezeigt** wird, liegt das am bekannten Geräte-name-Problem:
 - Boote das Live-System, öffne ein Terminal und führe aus:

```
sudo rm /dev/mmcblk0
```

(udev legt das Gerät danach sauber neu an.)

5. Wichtig – GRUB auf die UEFI-Wechselmedien-Position setzen:

- GRUB nicht auf die eMMC installieren lassen, sondern in den **EFI-Pfad**.
 - Nach der Installation feat den Abschnitt 4 (BOOTX64.EFI) beachten, sonst gibt es „No bootable devices found“.
6. Den Benutzer anlegen, unter dem MagicQ laufen soll:

```
sudo adduser chamsys
```

(Dieses Skript und der MagicQ-Autostart nutzen den Benutzer `chamsys`.)

7. Nach der Installation vom USB-Stick booten (siehe Abschnitt 4).

4. Boot-Pfad reparieren (BOOTX64.EFI) – unbedingt nötig!

Der Wyse 3040 startet **nur** von `\EFI\BOOT\BOOTX64.EFI`. Frisch installiertes Ubuntu/GRUB liefert aber `\EFI\debian\grubx64.efi` o. Ä. Ohne den Fallback-Pfad → „No bootable devices found“.

1. Boote mit dem Live-USB-Stick.
2. eMMC-Boot-Partition mounten:

```
sudo blkid /dev/mmcblk0p1      # sollte TYPE="vfat" zeigen

sudo mkdir -p /mnt/p1

sudo mount /dev/mmcblk0p1 /mnt/p1
```

3. Fallback anlegen:

```
sudo mkdir -p /mnt/p1/EFI/BOOT

sudo cp /mnt/p1/EFI/ubuntu/grubx64.efi /mnt/p1/EFI/BOOT/BOOTX64.EFI

# oder falls "debian":

sudo cp /mnt/p1/EFI/debian/grubx64.efi /mnt/p1/EFI/BOOT/BOOTX64.EFI

sudo umount /mnt/p1
```

4. USB-Stick entfernen und neu starten. Ubuntu sollte nun von der eMMC booten.

5. Setup-Skript ausführen

Wichtig: Das Skript muss als **root** (`sudo`) ausgeführt werden, weil es Systempakete installiert. MagicQ läuft aber unter dem Benutzer `chamsys`. Das Skript erstellt/verwendet alle Benutzerdateien daher gezielt unter `/home/chamsys` (nicht unter `/root`).

 **Voraussetzung:** Der Benutzer `chamsys` muss existieren, bevor du das Skript startest: `bash`

```
> sudo adduser chamsys
>
```

Das Skript `setup_magicq_wyse.sh` macht Folgendes automatisch:

- Legt **zuerst** `/home/chamsys/.config/openbox/` an (bevor darauf zugegriffen wird).
- Installiert Openbox (leichter Window-Manager), xinit, **xserver-xorg** (der eigentliche X-Server — ohne ihn scheitert `startx` mit `exec: /usr/bin/X: not found`), X-/USB-Basistools.
- Installiert **Qt5-, xcb- und Laufzeit-Abhängigkeiten** (inkl. fix für den Fehler „could not load QT plugin xcb“).
- Installiert MagicQ, falls eine `.deb` im selben Ordner liegt, und prüft die Bibliotheken per `ldd`.
- Richtet den Openbox-Autostart für `chamsys` ein:
 1. X automatisch startet,
 2. MagicQ im **Fullscreen / Panel-Modus für den Compact Wing** öffnet,
 3. MagicQ **ohne Fensterdekorationen** anzeigt.
- Setzt **alle Ethernet-Interfaces auf DHCP** (via netplan, `en*`).
- Richtet **Autologin** ein (`tty1` → direkt zu Openbox + MagicQ, ohne Passwort).

- Installiert **Plymouth-Splashscreen** (`splash.png`) und setzt GRUB auf `quiet splash` mit `GRUB_GFXMODE=1280x800` .
- Legt die Auflösung auf **1280x800** fest (GRUB/Splash **und** X11 via `/etc/X11/xorg.conf.d/11-resolution.conf` → MagicQ läuft fullscreen in 1280x800).
- Erstellt die nötigen Benutzer-/Systemdateien und setzt die Besitzer auf `chamsys` .

Skript ausführen:

```
chmod +x setup_magicq_wyse.sh
sudo ./setup_magicq_wyse.sh          # ohne .deb -> MagicQ später manuell
sudo ./setup_magicq_wyse.sh magicq_ubuntu_*.deb  # mit .deb im selben Ordner
```

6. MagicQ installieren (falls nicht per Skript erledigt)

1. Deutsche Download-Seite: <https://www.chamsys.co.uk/mqdownload/> → **Ubuntu (64 bit)** `.deb` herunterladen.
2. Installieren:

```
sudo dpkg -i magicq_ubuntu_*.deb
# ggf. Abhängigkeiten nachziehen:
sudo apt-get -f install
```

3. MagicQ wird nach `/opt/magicq/` installiert. Start von Hand zum Testen:

```
sudo -u chamsys /opt/magicq/runmagicq.sh
```

> **LibGL-Fehler** beim Start? Dann: > `bash`

```
> sudo mv /opt/magicq/lib/libstdc.so.6 /opt/magicq/lib/libstdc.so.6~
```

```
> > ggf. auch QT_AUTO_SCREEN_SCALE_FACTOR=0 in runmagicq.sh exportieren.
```

6.1 Qt5- und Laufzeit-Abhängigkeiten

Wichtig zu wissen:

- MagicQ **bündelt seine Qt5-Bibliotheken selbst** unter `/opt/magicq/lib` (`bin/mqqt` nutzt sie per `LD_LIBRARY_PATH`). Für den reinen Start sind also **keine** Qt5-Systempakete zwingend nötig.
- `sudo dpkg -i` + `apt-get -f install` zieht nur die im Paket *deklarierten* Depends nach. Nicht-Qt-Laufzeitbibliotheken (GLU, USB, PortAudio, FFmpeg, GStreamer, Alsa) sind dort teils **nicht** deklariert und können beim Start trotzdem fehlen.

Das Setup-Skript installiert daher automatisch **beide** Sicherheitsnetze:

Zweck	Pakete
Qt5 (Fallback + Multimedia)	<code>libqt5core5a libqt5gui5 libqt5widgets5 libqt5network5 libqt5opengl5 libqt5printsupport5 libqt5xml5 libqt5sql5 libqt5multimedia5 libqt5multimediawidgets5 libqt5svg5 libqt5qml5 libqt5quick5</code>
OpenGL / X11	<code>libglu1-mesa libgl1 libglx-mesa0 libxext6 libxrender1</code>
USB (Wings/DMX-Interfaces)	<code>libusb-1.0-0 libusb-0.1-4</code>
Audio / Video	<code>libportaudio2 libasound2* ffmpeg libgstreamer1.0-0 libgstreamer-plugins-base1.0-0 gstreamer1.0-plugins-base gstreamer1.0-plugins-good</code>

ALSA-Paketname (`libasound2*`): Variiert je nach Ubuntu-Version!

- **Ubuntu 22.04:** `libasound2`
- **Ubuntu 24.04+:** `libasound2t64` (das alte `libasound2` existiert dort nicht mehr → „libasound2 has no installation candidate“).

Das Skript wählt den korrekten Namen automatisch je nach Version. Auch `libglu1-mesa`, `libgl1` etc. sind auf 24.04+ teils als `t64`-Variante benannt; das Skript versucht die passende Auswahl und fängt Fehlschläge ab.

| Archive / Basis | `libarchive13 zlib1g libglib2.0-0 libstdc++6` |

Fehler „could not load QT plugin xcb“:

Dieser Fehler tritt auf, wenn das Qt5-xcb-Plattform-Plugin seine X11-Bibliotheken nicht findet. Das Skript installiert deshalb zusätzlich alle nötigen `libxcb*`-Pakete:

Zweck	Pakete
xcb-Plugin (Qt5)	<code>libxcb-xinerama0 libxcb-cursor0 libxcb-keysyms1 libxcb-image0 libxcb-render-util0 libxcb-icccm4 libxcb-shape0 libxcb-xfixes0 libxcb-xkb1 libxcb-xinput0 libxcb-randr0 libxcb-sync1 libxcb-shm0 libxcb1</code>
XKB / Fonts / EGL	<code>libxkbcommon-x11-0 libxkbcommon0 libfontconfig1 libfreetype6 libx11-xcb1 libegl1 libgl1 libglx-mesa0</code>

GL/Mesa-Paketnamen (`libgl1.*`): `libgl1-mesa-glx` existiert in ****Ubuntu 24.04+ nicht mehr**** (seit 23.10 entfernt, war schon lange nur ein Übergangs-Paket). Es wird durch `libgl1` **und** `libglx-mesa0` ersetzt. Das Skript nutzt daher immer `libgl1` + `libglx-mesa0` — das funktioniert auf 22.04 und 24.04 gleichermaßen.

Zusätzlich erzwingt das Start-Skript `/usr/local/bin/start_magicq.sh` die xcb-Plattform:

```
export QT_QPA_PLATFORM=xcb
export QT_PLUGIN_PATH=/opt/magicq/plugins
```

Manuell prüfen / nachrüsten:

```
sudo apt-get install -y --no-install-recommends \
    libqt5gui5 libxcb-xinerama0 libxcb-cursor0 libxcb-keysyms1 libxcb-image0 \
    libxcb-render-util0 libxcb-icccm4 libxcb-shape0 libxcb-xfixes0 libxcb-xkb1 \
    libxcb-xinput0 libxcb-randr0 libxcb-sync1 libxcb-shm0 libxcb1 \
    libxkbcommon-x11-0 libxkbcommon0 libfontconfig1 libfreetype6
```

7. Compact Wing & Panel-Modus – wie es funktioniert

Der Autostart startet MagicQ mit dem **Full-Panel-Modus** (imitiert ein Compact-Console-Layout), im **Fullscreen**. Der Compact Wing schaltet MagicQ in den **Unlocked-Modus** (voller DMX-Ausgang), sobald er per USB verbunden ist.

- **Panel-Modus wechseln:** In MagicQ → `Setup → View Settings → Panels` → **Full Panel**.
- **Wing prüfen:** `Setup → View Settings → Ports → MagicQ Wings & Interfaces = Yes (auto D MX)`.
- Der Wing braucht **keinen separaten Treiber** (nur die neueren Compact-Wings).
- Für alten PC/Extra-Wing (FTDI): `Setup → View Settings → Ports → FTDI + VCP driver`.

Hinweis zur Konfiguration: Der „Full-Panel-Modus“ und „Fullscreen“ werden im Skript über die Auto-Start-Kommandozeile und eine einmalige Konfig-Datei gesetzt. Falls MagicQ beim ersten Start nicht automatisch im gewünschten Modus startet, einmal die gewünschten Einstellungen speichern – MagicQ merkt sich den Zustand dann über Neustarts hinweg.

8. Autostart (was das Skript anlegt)

- `~/.config/openbox/autostart` – wird beim Openbox-Login ausgeführt und enthält:

```
# X-Server wird per .xinitrc / xinit gestartet (falls nicht schon läuft)
feh --bg-scale /usr/share/backgrounds/warty-final-ubuntu.png &
# USB-Sticks automatisch mounten (zusätzlich zu udev-Regen)
for d in /dev/sd*; do [ -b "$d" ] && udiskctl mount -b "$d" 2>/dev/null; done &
# MagicQ starten (Fullscreen / Panel-Modus Compact)
sleep 5
/usr/local/bin/start_magicq.sh &
```

- `/usr/local/bin/start_magicq.sh` – startet MagicQ mit dem richtigen Config-Argument.
- **udev-Regel** `/etc/udev/rules.d/99-magicq.rules` (optional, vom Skript angelegt) zum automatischen Mounten von USB-Sticks.

9. Keine Fensterdekorationen für MagicQ

Damit MagicQ **ohne Titel- und Fensterrahmen** (nur der Inhalt) den kompletten Bildschirm füllt, konfiguriert das Skript Openbox über `~/.config/openbox/rc.xml`. Das eingefügte Anwendungs-Regel-Fragment sieht so aus:

```
<applications>
  <application class="*/*MagicQ*">
    <decor>no</decor>
    <fullscreen>yes</fullscreen>
  </application>
</applications>
```

- `<decor>no</decor>` entfernt den Fensterrahmen (kein Schließen-/Minimieren-Button).
- `<fullscreen>yes</fullscreen>` stellt sicher, dass MagicQ den ganzen Bildschirm nutzt.

Manuell nachbearbeiten:

```
nano ~/.config/openbox/rc.xml
# Abschnitt <applications> prüfen/anpassen, speichern
openbox --reconfigure
```

Hinweis: Die Fensterklassen-Bezeichnung kann je nach MagicQ-Version abweichen. Falls die Regel nicht greift, im X-Terminal die Klasse prüfen: `bash`

```
> xprop | grep WM_CLASS      # nach Fokus auf das MagicQ-Fenster
> und den Wert in der class=-Zeile entsprechend anpassen.
```

10. Alle Ethernet-Interfaces auf DHCP

Das Skript legt eine netplan-Datei `/etc/netplan/99-magicq-dhcp.yaml` an, die **alle** Ethernet-Interfaces (Pattern `en*`) auf DHCP (IPv4 **und** IPv6) setzt:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    all-eth:
      match:
        name: "en*"
      dhcp4: yes
      dhcp6: yes
```

Nach Ausführung des Skripts einmalig aktivieren:

```
sudo netplan apply
ip a          # prüfen: Interfaces en* haben 192.168.x.x zugewiesen
```

Boot hängt ohne Netzwerk-Kabel: Das Skript **maskiert** zusätzlich den Service `systemd-networkd-wait-online.service`. Ohne diese Maske wartet `network-online.target` bei fehlendem Carrier (Kabel nicht angesteckt) **unbegrenzt** → der Boot bleibt stehen. Mit maskiertem Waiter bootet der Wyse sofort weiter, auch ohne Kabel; sobald ein Kabel eingesteckt wird, bekommt er weiterhin per DHCP eine IP (Netzwerk läuft im Hintergrund). MagicQ braucht kein Netzwerk zum Booten.

```
sudo systemctl mask systemd-networkd-wait-online.service
```

Der Wyse 3040 hat standardmäßig einen 1-Gbit-Ethernet-Port. Über einen USB-Ethernet-Adapter (für Light-Show/Art-Net/sACN über ein zweites Netz) wird dieser ebenfalls als `en*` erkannt und automatisch per DHCP konfiguriert. Falls du mehrere Netze trennen willst (Show-Netz vs. Art-Net), kannst du ein Interface per `networkd` / `netplan` mit statischer IP ergänzen – DHCP bleibt für die übrigen aktiv.

11. Automatisches Mounten von USB-Sticks

Das Skript wählt automatisch den passenden Weg **abhängig von der Ubuntu-Version**:

- **Ubuntu 22.04 und älter:** installiert `usbmount` (einfachste Lösung), das Sticks automatisch nach `/media/usb*` mountet.

- **Ubuntu 24.04 und neuer:** `usbmount` existiert dort nicht mehr – der Befehl `apt-get install usbmount` würde mit „Unable to locate package usbmount“ fehlschlagen. Das Skript überspringt `usbmount` auf diesen Versionen und nutzt stattdessen `udev + udisks` (siehe unten).

udev + udisks (Standard auf modernen Systemen bzw. Fallback):

Das Skript legt die Regel `/etc/udev/rules.d/99-magicq-usb.rules` an und setzt den Owner.

Zusätzlich mountet der Openbox-Autostart bereits eingesteckte Sticks beim Start:

```
for d in /dev/sd[b-z]*; do
    [ -b "$d" ] && udiskctl mount -b "$d" 2>/dev/null || true
done &
```

12. Autologin (optional, für „Booten direkt zu MagicQ“)

Das Setup-Skript richtet automatisch ein: Nach dem Einschalten wird auf `tty1` ohne Passwort der Benutzer eingeloggt, `startx` gestartet und Openbox + MagicQ im Fullscreen geöffnet – ganz ohne Interaktion.

Was das Skript dafür macht:

1. **systemd-Autologin** über eine Override-Datei `/etc/systemd/system/getty@tty1.service.d/autologin.conf`:

```
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin chamsys --noclear tty1 linux
```

Damit wird der Benutzer `chamsys` automatisch bei `tty1` eingeloggt.

2. `/home/chamsys/.bash_profile` wird um einen Start-Block ergänzt, der `startx` ausführt, sobald man auf `tty1` landet:

```
if [ -z "$DISPLAY" ] && [ "$(tty)" = "/dev/tty1" ]; then
    exec startx
fi
```

3. `/home/chamsys/.xinitrc` startet `openbox-session` (bereits aus Abschnitt 5), damit nach `startx` Openbox mit dem MagicQ-Autostart (Fullscreen, keine Dekorationen) erscheint.

Alle Dateien gehören dem Benutzer `chamsys` (`chown chamsys:chamsys`).

Manuell nachbauen oder auf anderen Benutzer umstellen:

```
sudo mkdir -p /etc/systemd/system/getty@tty1.service.d
sudo tee /etc/systemd/system/getty@tty1.service.d/autologin.conf >/dev/null <<EOF
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin chamsys --noclear tty1 linux
EOF
sudo systemctl daemon-reload
sudo systemctl enable getty@tty1.service
```

Deaktivieren (Login-Passwort wieder verlangen):

```
sudo rm /etc/systemd/system/getty@tty1.service.d/autologin.conf
sudo systemctl daemon-reload
```

13. Splashscreen (Boot-Logs verdecken)

Damit beim Booten **keine Kernel-/System-Logs**, sondern **dein Bild** `splash.png` fullscreen angezeigt werden, richtet das Skript **Plymouth** ein und setzt GRUB auf `quiet splash`.

⚠ Wichtig: `splash.png` muss sich im ****gleichen Ordner wie** `setup_magicq_wyse.sh` ****** befinden, wenn du das Skript ausführst – es wird dort automatisch gefunden und eingebunden.

Wie es funktioniert (robuster Ansatz):

Das Skript baut ein **eigenes Plymouth-Theme namens** `magicq-splash` auf Basis des `script`-Moduls. Das zeigt dein `splash.png` als Ganzbild über den gesamten Bildschirm.

⚠ Warum `script` und nicht `backgrounds`? Auf **Ubuntu 24.04 (noble)** gibt es das `backgrounds`-Plugin **in keinem Paket** (die Dateiliste `plymouth` / `plymouth-themes` enthält nur `script.so`, `text.so`, `tribar.so`, `details.so`, `fade-throbber.so`, `space-flares.so`). Ein früheres Thema auf `ModuleName=backgrounds` erzeugte deshalb beim Boot die Meldung „plugin `backgrounds.so` is missing“. Wir nutzen stattdessen `script.so`, das im **`plymouth`-Basis-Paket** (main, immer vorhanden) steckt – damit ist das Modul garantiert da. `label-pango.so` wird **nicht** gebraucht.

Ablauf:

1. Installiert `plymouth`, `plymouth-themes`, `plymouth-label`, `plymouth-theme-script`, `plymouth-theme-spinner`, `plymouth-theme-ubuntu-logo`, `plymouth-theme-ubuntu-text` (Absicherung).
2. Legt `/usr/share/plymouth/themes/magicq-splash/` an und kopiert `splash.png` → `background.d.png` (Fallback: das `ubuntu-logo`-Logo).

3. Schreibt eine `magicq-splash.plymouth`-Datei mit `ModuleName=script` **plus** eine `magicq-splash.script`-Datei (der `script`-Nutzcode, der das Bild über die volle Bildschirmfläche skaliert).
4. **Aktiviert das Theme als Standard.** Auf Ubuntu 24.04 (Noble) fehlt das Binary `plymouth-set-default-theme` (Ubuntu-Bug LP #1596220 – `/usr/bin` enthält nur `plymouth`), und `update-alternatives --set` meldet oft fälschlich Erfolg. Das Skript setzt deshalb den Alternatives-Link `/etc/alternatives/default.plymouth` **direkt** auf unser Theme und **verifiziert** das Ergebnis per `readlink`.
5. Schreibt in `/etc/default/grub`:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash loglevel=3 vt.global_cursor_default=0"
GRUB_GFXMODE=1280x800
GRUB_GFXPAYLOAD_LINUX=keep
GRUB_TERMINAL_OUTPUT=console
GRUB_TIMEOUT=0
```

- `quiet splash` (Pflicht!) → aktiviert den Splash + keine Kernel-Meldungen.
 - `loglevel=3` → nur Fehler/Warnungen auf Konsole (praktisch nichts).
 - `GRUB_GFXMODE=1280x800` / `GRUB_GFXPAYLOAD_LINUX=keep` → Grafikkonsole + Splash laufen in **1280x800**, damit der Splash fullscreen läuft (verhindert Umschalten auf Text-VGA).
 - `GRUB_TERMINAL_OUTPUT=console` → verhindert, dass GRUB die Ausgabe auf einen fremden Terminal zwingt.
6. Führt `update-initramfs -u` und `update-grub` aus und **prüft danach** per `lsinitramfs`, ob Theme + `script.so` in der initramfs gelandet sind. Fehlt etwas, wird das Theme erneut gesetzt und neu gebaut.

Aktivierung prüfen / nachträglich setzen (ohne `plymouth-set-default-theme`):

```
readlink /etc/alternatives/default.plymouth          # muss auf magicq-splash zeigen

# Falls noch nicht aktiv, direkt auf unser Theme zeigen lassen:
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
    /etc/alternatives/default.plymouth
sudo update-initramfs -u
sudo update-grub
```

Bild nachträglich austauschen:

```
sudo cp /neuer/pfad/splash.png /usr/share/plymouth/themes/magicq-splash/background.png
sudo update-initramfs -u
sudo update-grub
```

Splash testen / prüfen:

```
# Plymouth läuft?
sudo plymouth --ping && echo "Plymouth läuft"

# Unser Theme eingebacken?
lsinitramfs /boot/initrd.img-$(uname -r) | grep -i magicq-splash

# script-Modul (aus plymouth-Basis) vorhanden?
ls /usr/lib/x86_64-linux-gnu/plymouth/script.so

# GRUB-Parameter korrekt?
cat /boot/grub/grub.cfg | grep -i "quiet splash"
```

13.1 Fehlerbehebung: „nur schwarzer Bildschirm statt Splash“

Ein schwarzer Bildschirm statt des Splashes hat fast immer eine dieser Ursachen — prüfe sie in dieser Reihenfolge:

1. **splash fehlt in den Kernelparametern** → ohne **splash** startet Plymouth gar nicht (nur leere Konsole = schwarz).

```
sudo nano /etc/default/grub

# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash ..." <- splash MUSS drin sein

sudo update-grub
```

2. **Script-Modul fehlt** → wenn ein **ModuleName=script**-Theme genutzt wird und **plymouth-theme-script** nicht installiert ist, fehlt **script.so** und das Theme lädt nicht.

```
sudo apt-get install -y plymouth-theme-script

sudo update-initramfs -u
```

(Das Skript installiert dieses Paket jetzt automatisch.)

3. **label-Modul fehlt / nicht im initramfs** → „the plugin label-pango.so is missing“. Das **label**-Modul liegt im Paket **plymouth-label** (bzw. **plymouth-themes**). **Wichtig:** Es reicht nicht, das Paket nur zu installieren — die **.so**-Datei muss zusätzlich in die **initramfs** eingebettet werden (das ist die häufigste Ursache dafür, dass der Fehler trotz Installation weiter auftritt).

```
sudo apt-get install -y plymouth-themes plymouth-label

sudo update-initramfs -u          # MUSS nach der Installation laufen

# Kontrolle, ob es eingebettet wurde:

lsinitramfs /boot/initrd.img-$(uname -r) | grep -i "label-pango|label.so"
```

(Das Skript vermeidet dieses Problem von vornherein: Es nutzt ein eigenes `script`-Theme, das gar kein `label`-Modul braucht.)

4. `plymouth-set-default-theme: command not found` → auf **Ubuntu 24.04 (Noble)** fehlt dieses Binary im `plymouth`-Paket bekanntermaßen (Ubuntu-Bug LP [#1596220](#)); `/usr/bin` enthält dann nur `plymouth`. Das ist kein Installationsfehler. Das Default-Theme setzt man in dem Fall über `update-alternatives`:

```
sudo update-alternatives --config default.plymouth
sudo update-initramfs -u
```

(Das Skript erkennt das und aktiviert das Theme automatisch via `update-alternatives --set default.plymouth` bzw. Symlink — es verlässt sich nicht mehr auf das fehlende Kommando.)

4. **Theme fehlt in der initramfs** → `update-initramfs -u` wurde vergessen bzw. fehlgeschlagen.

```
sudo update-initramfs -u
lsinitramfs /boot/initrd.img-$(uname -r) | grep -i "ubuntu-logo|magicq"
```

5. **Grafik-Treiber**/`nomodeset` → Wenn `nomodeset` in den Kernelparametern steht, wird Plymouth oft nicht angezeigt. Entferne `nomodeset` (außer es ist aus anderen Gründen zwingend nötig).

6. **Falsche Auflösung** → Das Skript setzt `GRUB_GFXMODE=1280x800` (Boot/Splash) und erzwingt über `/etc/X11/xorg.conf.d/11-resolution.conf` auch in X11 1280x800. Wenn der Monitor eine andere native Auflösung hat, passe beide Werte an.

7. **Display-Hardware** → Am Wyse 3040 ist ein **aktiver DP→HDMI-Adapter** nötig. Ohne passenden Adapter bleibt der Bildschirm u. U. schwarz.

8. **Splash flackert / wird immer wieder schwarz** → Bekannter Plymouth-`script`-Bug: Der Framebuffer wird bei jedem Refresh-Zyklus geleert, und wenn der Theme-Callback das Bild nicht **jedes Mal neu setzt**, ist der Screen wiederholt schwarz. Das Skript redrawet das Bild jetzt in jedem Refresh (siehe `magicq-splash.script`: `draw_bg()` mit `SetPosition` / `SetScale` / `SetOpacity` / `SetZ(15)` im `refresh_callback`). Nach Änderungen:

```
sudo update-initramfs -u && sudo reboot
```

*(Ein kurzer schwarzer Blitz **genau einmal** kurz vor dem X-Desktop ist dagegen der normale Übergang Plymouth → Display-Manager und kein Fehler.)*

9. **Es erscheint ein (Ubuntu-/Hersteller-)Logo statt `splash.png`** → `/etc/alternatives/default.plymouth` zeigt noch auf ein anderes Theme (z. B. `bgrt` oder `ubuntu-logo`). Das Aktivieren über `update-alternatives --set` greift auf Noble oft nicht. Direkt auf unser Theme zeigen lassen und neu bauen:

```
readlink /etc/alternatives/default.plymouth
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
    /etc/alternatives/default.plymouth
sudo update-initramfs -u && sudo update-grub
sudo reboot
```

Boot-Logs überhaupt anzeigen (nur bei Diagnose):

Halt beim Booten eine **Shift**-Taste (bzw. **Esc** in GRUB) gedrückt, um das GRUB-Menü zu öffnen und die **quiet splash**-Kernelparameter über **e** zu entfernen – dann erscheinen wieder alle Meldungen.

13.2 Fehlerbehebung: „exec: /usr/bin/X: not found“

Dieser Fehler erscheint beim Autologin/**startx** aus **/etc/X11/xinit/xserverrc** und bedeutet, dass **der X-Server selbst nicht installiert ist**. **xinit**/**startx** brauchen **xserver-xorg**, das für den **/usr/bin/X**-Startpfad sorgt.

Behebung:

```
sudo apt-get install -y xserver-xorg
sudo update-alternatives --install /usr/bin/X x-session-manager /usr/bin/Xorg 50
startx
```

Kontrolle:

```
ls -la /usr/bin/X          # muss auf /usr/bin/Xorg zeigen
which Xorg
```

(Das Skript installiert **xserver-xorg** jetzt automatisch in Schritt 1.)

13.3 Fehlerbehebung: MagicQ „cannot create the data folder“

MagicQ speichert Show-Dateien und Einstellungen in **~/MagicQ** bzw. unter **~/.config/MagicQ** und **~/.local/share/Chamsys**. Fehlen diese oder sind sie nicht beschreibbar, meldet MagicQ, es könne das Datenverzeichnis nicht anlegen.

Manuelle Behebung (als root):

```

sudo mkdir -p /home/chamsys/MagicQ

sudo mkdir -p /home/chamsys/.config/MagicQ

sudo mkdir -p /home/chamsys/.local/share/Chamsys

sudo chown -R chamsys:chamsys /home/chamsys

ls -ld /home/chamsys/MagicQ    # muss chamsys gehören

```

(Das Skript legt das Datenverzeichnis in Schritt 5e automatisch an und erzwingt `$HOME` im Start-Skript, damit MagicQ den richtigen Ort findet.)

13.4 Maus-Cursor ausblenden

Für den Show-PC soll kein Mauszeiger sichtbar sein (weder im Openbox-Desktop noch in der Qt-Anwendung MagicQ). Das Skript erzeugt ein **gültiges transparentes XCursor-Theme** im **XCursor-Binärformat** und legt es **systemweit** ab:

1. Baut das Theme `Transparent` in `/usr/share/icons/Transparent` (immer gefunden, unabhängig von `$HOME`) und spiegelt es zusätzlich nach `~/.icons/Transparent`.
2. Setzt `/usr/share/icons/default` → `Transparent` (der **globale X-Default**):
Selbst wenn Qt/MagicQ einen Cursor-Namen anfordert, löst Xcursor ihn System-weit gegen unser transparentes Theme auf — das ist der entscheidende Fix für den Zeiger ****über dem MagicQ-Fenster**** (vorher lag das Theme nur im Home-Verzeichnis und wurde bei abweichendem `$HOME`/anderem Suchpfad nicht gefunden).
3. Trägt `XCURSOR_THEME` / `XCURSOR_SIZE` global in `/etc/environment` ein und setzt in `~/.config/gtk-3.0/settings.ini` `gtk-cursor-theme-name`, damit Qt **und** GTK das Theme nutzen.

⚠ Warum reines Binärformat nötig ist: Die früheste Version legte **rohe PNG-Dateien** unter `cursors/` ab — Xcursor verlangt aber das **XCursor-Dateiformat** (`Xcur`-Header) und ignorierte PNGs stillschweigend (Pfeil blieb sichtbar). Das Skript schreibt die `.cursor`-Datei **direkt per Python** (exakt nach `XcursorXcFileSave` aus `libXcursor`: Magic `0x72756358`, Version `0x00010000`, ein `32×32-Image-Chunk` mit `type=0xfffd0002`, volltransparente ARGB-Pixel) — ganz **ohne** externe Tools. `xcursorgen` (aus `x11-apps`) dient nur noch als Fallback, falls `python3` fehlt. Damit sind die Dateien garantiert **>0 Byte** und von Xcursor **und** Qt ladbar.

Zusätzlich:

- setzt das Skript `xsetroot -cursor_name none` in `~/.xinitrc` **vor** `exec openbox-session` (Root-Zeiger unsichtbar) und in `/usr/local/bin/start_magicq.sh` erneut `XCURSOR_THEME` / `XCURSOR_SIZE` + `xsetroot`, damit der transparente Zeiger direkt im MagicQ-Prozess greift.

`x11-apps` (liefert `xcursorgen`) liegt im universe-Repo; es wird nur als **Fallback** gebraucht — der Python-Writer kommt ohne es aus. Manuell prüfen:

```
ls -l /usr/share/icons/Transparent/cursors/default    # MUSS >0 Byte sein (4160 Byte für
32×32)
xxd /usr/share/icons/Transparent/cursors/default | head -1    # beginnt mit: 5863 7572
("Xcur")
cat /usr/share/icons/default/index.theme              # Inherits=Transparent
cat /etc/environment                                  # XCURSOR_THEME=Transparent
```

13.5 Monitor soll immer an bleiben (kein Energiesparmodus)

Damit der Bildschirm am Show-PC ****nie**** in den Standby/Sparmodus geht, deaktiviert das

Skript im X-Start (Openbox-Autostart ****und**** in ``start_magicq.sh``):

```
```bash
xset s off # X-Screensaver aus
xset -dpms # DPMS (Standby/Suspend/Off) komplett aus
xset dpms 0 0 0 # Timeouts auf "nie"
setterm -blank 0 -powersave off -powerdown 0 # text-Konsole nie ausblenden
```

### Manuell auf der Box (im laufenden X):

```
xset s off
xset -dpms
xset dpms 0 0 0
```

*Wenn der Monitor trotzdem aus geht, prüfe zusätzlich die eigenen Monitor-Einstellungen (Auto-Standby) und ob der Verstärker/DP-Adapter dies steuert.*

## 14. Fehlerbehebung (Kurzliste)

Problem	Lösung
„No bootable devices found“	Abschnitt 4 (BOOTX64.EFI) befolgen
Installer findet eMMC nicht	<code>sudo rm /dev/mmcblk0</code> im Live-System
Installer friert ein	Neuere Ubuntu-Version meiden; Go to 24.04 LTS
MagicQ LibGL-Fehler	<code>libstdc.so.6</code> umbenennen (Abschnitt 6)
UI zu klein/verzerrt	<code>QT_AUTO_SCREEN_SCALE_FACTOR=0</code> in <code>runmagicq.sh</code>
Wing wird nicht erkannt	<code>Ports → MagicQ Wings &amp; Interfaces = Yes (auto DMX)</code>
Kein Bild auf HDMI	Aktiven DP→HDMI-Adapter verwenden
MagicQ braucht Root für USB	Skript startet MagicQ ggf. mit <code>sudo</code> / setzt passende udev-Regel
MagicQ hat noch Fensterrahmen	<code>~/.config/openbox/rc.xml</code> prüfen + <code>openbox --reconfigure</code> (Abschnitt 9)
<code>exec: /usr/bin/X: not found</code>	<code>sudo apt-get install xserver-xorg</code> (Abschnitt 13.2)
MagicQ: „cannot create the data folder“	<code>~/.config/MagicQ/</code> / <code>~/MagicQ</code> fehlen/nicht beschreibbar; mit <code>chown -R cham sys:chamsys</code> + <code>mkdir</code> anlegen (Abschnitt 5e)
XML-Syntaxfehler in <code>~/.config/openbox/rc.xml</code>	<code>rc.xml</code> kaputt/leer → Skript schreibt sie jetzt vollständig neu; manuell <code>sudo apt-get install --reinstall openbox</code> oder Datei löschen, damit sie neu erzeugt wird
Ethernet bekommt keine IP	<code>sudo netplan apply</code> + <code>ip a</code> prüfen (Abschnitt 10)
Boot-Logs erscheinen trotz Splash	GRUB <code>quiet splash</code> prüfen (Abschnitt 13)
Ubuntu-Logo statt eigenem Splash	<code>/etc/alternatives/default.plymouth</code> direkt auf <code>magicq-splash</code> verlinken + <code>update-initramfs</code> (Abschnitt 13.1 Punkt 8; oft liegt das <code>bgrt</code> -Theme zugrunde)
Monitor geht in den Standby/Sparmodus	<code>xset -dpms</code> , <code>xset s off</code> , <code>xset dpms 0 0 0</code> im X-Start (Abschnitt 13.5)
Mauscursor sichtbar (soll unsichtbar sein)	Skript schreibt gültiges transparentes Theme im XCursor-Binärformat direkt per Python ( <code>Xcur</code> , 4 KB-Datei) systemweit + <code>/usr/share/icons/default</code> → <code>Transparent</code> (Abschnitt 13.4). Prüfe: <code>ls -l /usr/share/icons/Transparent/cursors/left_ptr</code> muss <b>&gt;0 Byte</b> sein (4160)
Schwarzer Bildschirm statt Splash	Abschnitt 13.1 (script-Modul, <code>splash</code> -Parameter, <code>nomodeset</code> , Auflösung, DP-Adapter)
Missing shared library / Qt5-Fehler	<code>cd /opt/magicq &amp;&amp; LD_LIBRARY_PATH=/opt/magicq/lib ldd ./bin/mqqt</code> + <code>sudo apt-get -f install</code> (Abschnitt 6.1)

Problem	Lösung
„could not load QT plugin xcb"	<code>libxcb-*</code> -Pakete installieren, <code>QT_QPA_PLATFORM=xcb</code> setzen (Abschnitt 6.1)
Kein Bild / Blackscreen nach Boot	Aktiven DP→HDMI-Adapter verwenden, <code>QT_SCREEN_SCALE_FACTORS=1</code> prüfen

---

## 15. Kompakte Befehlsübersicht

```
MagicQ manuell starten
/opt/magicq/runmagicq.sh

Autostart-Datei bearbeiten
nano ~/.config/openbox/autostart

USB-Mount testen
udisksctl mount -b /dev/sdb1
ls /media/ # Mountpunkt für usbmount

Openbox-Konfig (Fensterdekorationen) neu laden
openbox --reconfigure
nano ~/.config/openbox/rc.xml

DHCP aller Ethernet-Interfaces aktivieren
sudo netplan apply
ip a

Autologin deaktivieren (wieder Passwort verlangen)
sudo rm /etc/systemd/system/getty@tty1.service.d/autologin.conf
sudo systemctl daemon-reload

Splashscreen-Theme neu einspielen / GRUB aktualisieren
(auf Ubuntu 24.04 gibt es kein plymouth-set-default-theme; Link direkt setzen)
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
 /etc/alternatives/default.plymouth
sudo update-initramfs -u
sudo update-grub

Maus-Cursor unsichtbar machen (paketloser Fallback; Theme wird vom Skript im Home erzeugt)
sudo mkdir -p /usr/share/icons/default
sudo sh -c 'echo -e "[Icon Theme]\nInherits=Transparent" > /usr/share/icons/default/index.theme'
echo 'xsetroot -cursor_name none' >> ~/.xinitrc

Monitor soll nie in den Sparmodus/Standby gehen (im X-Server)
xset s off
xset -dpms
xset dpms 0 0 0
```

```
Neustart
sudo reboot
```